

Back To Practice

Q1

Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

Select Language : C (GCC 9.2.0)

```
23 RT[i] -- tq;
24 } else {
25     current_time += RT[i];
26     RT[i] = 0;
27     CT[i] = current_time;
28     completed++;
29 }
30 }
31 }
32 if(executed == 0)
33     current_time++;
34 }
35 for(i = 0; i < n; i++) {
36     TAT[i] = CT[i] - AT[i];
37     WT[i] = TAT[i] - BT[i];
38     totalTAT += TAT[i];
39     totalWT += WT[i];
40 }
41 printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);
42 printf("Average Waiting Time: %.2f\n", totalWT / n);
43 return 0;
44 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

3
2Enter number of processes:
Enter Time Quantum:

Back To Practice

Q1

Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main() {
3     int n, tq, i;
4     printf("Enter number of processes: \n");
5     scanf("%d", &n);
6     printf("Enter Time Quantum: \n");
7     scanf("%d", &tq);
8     int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
9     for(i = 0; i < n; i++) {
10         printf("Enter AT and BT for P%d: \n", i + 1);
11         scanf("%d %d", &at[i], &bt[i]);
12         rt[i] = bt[i];
13     }
14     int completed = 0, current_time = 0;
15     float totalTAT = 0, totalWT = 0;
16     while(completed < n) {
17         int executed = 0;
18         for(i = 0; i < n; i++) {
19             if(at[i] <= current_time && rt[i] > 0) {
20                 executed = 1;
21                 if(rt[i] > tq) {
22                     current_time += tq;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

3
2Enter number of processes:
Enter Time Quantum: